

Lecture 7: Strong Induction and Fast Exponentiation

September 15, 2026

Lecturer and scribe: Karthik Gajulapalli

Last lecture, we used induction to prove statements about all natural numbers. Today, we allow the induction hypothesis to include all smaller inputs. We use this form of induction to prove that prime factorizations exist, then design and analyze a fast algorithm for computing powers. We begin with the logarithms that will appear in the analysis.

1 A Primer on Logarithms

1.1 A Logarithm Asks for an Exponent

For a base $b > 1$ and a number $x > 0$, the statement

$$\log_b x = y \iff b^y = x$$

defines the **logarithm of x to base b** . Read $\log_b x$ as “the exponent to which we raise b to get x .” For example, $\log_2 8 = 3$ because $2^3 = 8$, and $\log_{10} 100 = 2$ because $10^2 = 100$.

The **binary logarithm** is the logarithm to base 2. In these notes, an unqualified log means $\log_2$.

Key identity: exponentiation and logarithms undo each other. Since $\log_2 n$ is the exponent that produces n , we have

$$2^{\log_2 n} = n \quad (n > 0).$$

For example, $2^{\log_2 8} = 2^3 = 8$. The identity also holds when the logarithm is not an integer: $2^{\log_2 13} = 13$ exactly. In the other direction, $\log_2(2^k) = k$.

x	1	2	4	8	16	32	1024
$\log_2 x$	0	1	2	3	4	5	10

Logarithms need not be integers: since $2^3 < 13 < 2^4$, we have $3 < \log_2 13 < 4$ (approximately 3.70). Also, $\log_2(1/2) = -1$. The real logarithm is undefined at 0 and at negative inputs.

1.2 Rules for Working with Logs

For $x, y > 0$ and a real number r ,

$$\begin{aligned} \log_b(xy) &= \log_b x + \log_b y, \\ \log_b(x/y) &= \log_b x - \log_b y, \\ \log_b(x^r) &= r \log_b x. \end{aligned}$$

These are exponent rules in reverse. For instance, if $x = b^u$ and $y = b^v$, then $xy = b^{u+v}$, so $\log_b(xy) = u + v$. In particular,

$$\log_2(2x) = 1 + \log_2 x, \quad \log_2(x/2) = \log_2 x - 1.$$

There is no corresponding rule that splits a sum: $\log_2(4+4) = 3$, whereas $\log_2 4 + \log_2 4 = 4$.

1.3 Inequalities, Rounding, and Repeated Halving

For $b > 1$, larger inputs have larger logarithms. Thus we can apply $\log_b$ to both sides of an inequality between positive numbers without reversing the inequality.

The notation $\lfloor t \rfloor$ means the greatest integer at most t , and $\lceil t \rceil$ means the least integer at least t . For an integer $n \geq 1$, there is a unique integer $k \geq 0$ with

$$2^k \leq n < 2^{k+1}.$$

Taking binary logarithms gives $k \leq \log_2 n < k + 1$, so

$$k = \lfloor \log_2 n \rfloor.$$

This is often more useful than a decimal approximation to a logarithm. For example, $8 \leq 13 < 16$ immediately gives $\lfloor \log_2 13 \rfloor = 3$.

Why do logarithms appear in algorithms? If we divide a quantity n by 2 a total of t times, the result is $n/2^t$. To reach a value at most 1, we need

$$\frac{n}{2^t} \leq 1 \iff 2^t \geq n \iff t \geq \log_2 n.$$

For $n \geq 1$, the smallest such integer t is $\lceil \log_2 n \rceil$.

Our algorithm will instead use *integer halving*, replacing n by $\lfloor n/2 \rfloor$, and stop when it reaches 1. Starting from an integer $n \geq 1$, this takes exactly $\lfloor \log_2 n \rfloor$ steps. For example,

$$13 \longrightarrow 6 \longrightarrow 3 \longrightarrow 1 \quad (\text{three integer halvings}).$$

Indeed, after t steps the value is $\lfloor n/2^t \rfloor$. If $2^k \leq n < 2^{k+1}$, then at $t = k$ this value is 1; at every earlier step it is at least 2.

Changing the base. For any other base $c > 1$,

$$\log_b n = \frac{\log_c n}{\log_c b}.$$

Changing between fixed bases only multiplies the logarithm by a constant. This is why an $O(\log n)$ bound usually does not specify a base. For exact counts, however, the base matters; we will use 2.

2 Strong Induction and Prime Factorization

In ordinary induction, we assume $P(i)$ to prove $P(i + 1)$. In **strong induction**, we may assume the statement for *every smaller input* in the domain. To prove $P(n)$ for all integers $n \geq n_0$:

1. Prove the base case $P(n_0)$.
2. For an arbitrary $n > n_0$, assume $P(m)$ holds for every integer m with $n_0 \leq m < n$, and use these assumptions to prove $P(n)$.

We still do not assume $P(n)$ itself. Strong induction is useful when a problem reduces to smaller inputs other than just $n - 1$.

This principle follows from ordinary induction by using the statement $Q(i)$: “ $P(m)$ holds for every m from n_0 through i .” The strong inductive step supplies the new fact $P(i + 1)$, which, together with $Q(i)$, gives $Q(i + 1)$.

2.1 Every Integer Greater Than One Is a Product of Primes

An integer $p \geq 2$ is **prime** if its only positive divisors are 1 and p . An integer $n \geq 2$ that is not prime is **composite**; it can be written as $n = ab$ with $1 < a, b < n$.

Claim 1. *Every integer $n \geq 2$ can be written as a product of one or more primes.*

Proof by strong induction. **Base case** ($n = 2$). The number 2 is prime, so it is already a product consisting of one prime.

Inductive step. Fix an integer $n > 2$ and assume that every integer m with $2 \leq m < n$ is a product of primes. There are two cases.

- If n is prime, it is itself the required product.
- If n is composite, write $n = ab$ with $2 \leq a, b < n$. The induction hypothesis applies to *both* factors, so

$$a = p_1 \cdots p_r, \quad b = q_1 \cdots q_s$$

for primes $p_1, \dots, p_r, q_1, \dots, q_s$. Therefore

$$n = ab = p_1 \cdots p_r q_1 \cdots q_s$$

is a product of primes.

Both cases establish $P(n)$. Strong induction proves the claim. □

Example. The proof suggests repeatedly splitting a composite factor:

$$84 = 7 \cdot 12 = 7 \cdot 3 \cdot 4 = 7 \cdot 3 \cdot 2 \cdot 2 = 2^2 \cdot 3 \cdot 7.$$

Each factor we split is replaced by smaller positive integers greater than 1. The induction hypothesis is useful because it covers these smaller factors, even though they need not equal $n - 1$.

3 Designing an Algorithm for Exponentiation

Given a positive integer a and a nonnegative integer n , we want to compute a^n . We use the convention $a^0 = 1$. Our goal is to **minimize the number of multiplications performed**: how can we compute the same value a^n using fewer multiplications?

3.1 The Natural Algorithm

For $n \geq 1$, the definition of exponentiation gives

$$a^n = \underbrace{a \cdot a \cdots a}_{n \text{ factors}}.$$

Start with one copy of a and multiply by a another $n - 1$ times:

```
SlowPower(a, n):
    if n == 0:
        return 1
    result = a
    for i = 2, 3, ..., n:
        result = result * a
    return result
```

When $n = 1$, the loop is empty. For $n \geq 1$ the algorithm uses exactly $n - 1$ multiplications. Starting instead with `result = 1` and multiplying n times uses exactly n multiplications. Either version uses a number of multiplications proportional to n .

If each multiplication is charged a cost C_{mult} , the multiplication work is $(n - 1)C_{\text{mult}}$ for the version above, or nC_{mult} for the second version.

3.2 Use the Same Smaller Power Twice

The identity $a^{2m} = (a^m)^2$ gives a better plan: compute a^m *once*, save it, and square it. For an odd exponent, there is one extra factor of a :

$$a^n = \begin{cases} (a^{\lfloor n/2 \rfloor})^2, & n \text{ even,} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a, & n \text{ odd.} \end{cases}$$

In either case, the recursive subproblem has exponent $\lfloor n/2 \rfloor$. This gives **fast exponentiation**, also called **exponentiation by squaring**.

```

FastPower(a, n):
    if n == 0:
        return 1
    if n == 1:
        return a

    y = FastPower(a, floor(n / 2))
    if n is even:
        return y * y
    else:
        return (y * y) * a

```

Each non-base call makes *one* recursive call. Its remaining work is one multiplication if n is even and two if n is odd. Squaring y counts as a multiplication.

Saving y matters. Writing two separate calls to `FastPower(a, n / 2)` in the even case would recompute the same answer twice. For powers of two, that version would satisfy $D(n) = 2D(n/2) + 1$, with $D(1) = 0$, giving $D(n) = n - 1$ multiplications and losing the improvement.

3.3 Correctness by Strong Induction

An algorithm must both terminate and return the right answer. We prove these properties together.

Claim 2. *For every positive integer a and every integer $n \geq 0$, `FastPower(a, n)` terminates and returns a^n .*

Proof. Fix an arbitrary positive integer a . We use strong induction on n .

Base cases. If $n = 0$, the algorithm immediately returns $1 = a^0$. If $n = 1$, it immediately returns $a = a^1$.

Inductive step. Let $n \geq 2$ and assume that for every integer m with $0 \leq m < n$, the call `FastPower(a, m)` terminates and returns a^m . Set $q = \lfloor n/2 \rfloor$. Since $1 \leq q < n$, the induction hypothesis applies to the one recursive call: it terminates and returns $y = a^q$.

- If n is even, then $n = 2q$. The algorithm returns

$$y \cdot y = a^q \cdot a^q = a^{2q} = a^n.$$

- If n is odd, then $n = 2q + 1$. The algorithm returns

$$(y \cdot y) \cdot a = a^q \cdot a^q \cdot a = a^{2q+1} = a^n.$$

After the recursive call, only finitely many operations remain, so the current call also terminates. This proves both properties for n . Strong induction completes the proof. $\square$

3.4 Counting Multiplications

Consider an odd exponent $n = 2m + 1 \geq 3$, where $m = \lfloor n/2 \rfloor$. There are two equal chunks of m factors and one extra factor of a :

$$a^n = \underbrace{a \cdot a \cdots a}_{m \text{ factors}} \overset{\cdot}{\uparrow} \underbrace{a \cdot a \cdots a}_{m \text{ factors}} \overset{\cdot}{\uparrow} a$$

Mult 1 Mult 2

We calculate one chunk, save its value $y = a^m$, and use that value twice. The two multiplications after the recursive call are: **Mult 1** joins the two equal chunks, and **Mult 2** multiplies by the extra factor of a . These give the $+2$. Because we save and reuse y , we pay for only one recursive call. Suppressing rounding, this contributes $M(n/2)$, **not** $2M(n/2)$. Two uses of a saved value do not require two recursive computations.

For a simple upper bound, charge two multiplications at every non-base call, even when the exponent is even and only one is needed. Let $M(n)$ denote this charged count. Suppressing integer rounding, we write

$$\boxed{M(n) = M(n/2) + 2, \quad M(1) = 0.}$$

This model bounds the actual number of multiplications from above. The input $n = 0$ is a separate base case requiring no multiplications.

3.4.1 Prove the Bound by Induction

Claim 3. *For every integer $n \geq 1$, the charged count satisfies $M(n) \leq 2 \log_2 n$.*

Proof by strong induction. **Base case** ($n = 1$). We have $M(1) = 0 = 2 \log_2 1$.

Inductive step. Let $n \geq 2$ and assume the bound holds for every smaller positive integer. To account for integer rounding, set $q = \lfloor n/2 \rfloor$. Then $1 \leq q < n$ and $q \leq n/2$. Applying the induction hypothesis to the recursive call,

$$\begin{aligned} M(n) &= M(q) + 2 \\ &\leq 2 \log_2 q + 2 && \text{(induction hypothesis)} \\ &\leq 2 \log_2 (n/2) + 2 && \text{(since } q \leq n/2) \\ &= 2(\log_2 n - 1) + 2 \\ &= 2 \log_2 n. \end{aligned}$$

This proves the bound for n , completing the induction. □

Thus the algorithm performs at most $2 \log_2 n$ multiplications. In particular, its multiplication count is $O(\log n)$.

3.4.2 Unroll the Recurrence

We can also see where the logarithm comes from by repeatedly substituting the recurrence into itself. Suppose n is a power of two, so there is no rounding:

$$\begin{aligned} M(n) &= M(n/2) + 2 \\ &= M(n/4) + 4 \\ &= M(n/8) + 6 \\ &\vdots \\ &= M(n/2^t) + 2t. \end{aligned}$$

We reach the base case when $n/2^t = 1$, or equivalently $2^t = n$. Thus $t = \log_2 n$. Using the primer's identity $2^{\log_2 n} = n$,

$$M(n) = M\left(\frac{n}{2^{\log_2 n}}\right) + 2 \log_2 n = M(1) + 2 \log_2 n = 2 \log_2 n.$$

For any positive integer n , rounding down gives $\lfloor \log_2 n \rfloor$ halving steps, so the charged count is $2 \lfloor \log_2 n \rfloor \leq 2 \log_2 n$, consistent with our induction proof. The key idea is **constant work per step, with only logarithmically many steps**. We will spend more time with recursion and recurrences later in the course.

From multiplication counts to time. One of the most important applications of fast exponentiation is **cryptography**, where computations such as $a^n \bmod N$ arise. Here n is the exponent and $N \geq 2$ is a fixed modulus. We reduce the base modulo N at the start and perform every multiplication modulo N :

multiply x and y , then keep $(xy) \bmod N$.

Thus every stored result lies between 0 and $N - 1$; even the temporary product is less than N^2 . The numbers never grow to the full size of a^n . **With the modulus fixed, our focus is the number of modular multiplications performed.** If each modular multiplication, including the reduction, is charged a cost C_{mult} , fast exponentiation uses $O(C_{\text{mult}} \log n)$ multiplication work, compared with $O(C_{\text{mult}} n)$ for the natural algorithm.

3.5 Binary Expansion and Repeated Squaring

Every positive integer n has a binary expansion

$$n = (b_k b_{k-1} \cdots b_1 b_0)_2 = \sum_{i=0}^k b_i 2^i, \quad b_i \in \{0, 1\}, \quad b_k = 1.$$

The rightmost bit is the coefficient of $2^0 = 1$, the next bit is the coefficient of $2^1 = 2$, and so on. Since $2^k \leq n < 2^{k+1}$, we have $k = \lfloor \log_2 n \rfloor$, and the binary representation has $k + 1$ bits.

For example,

$$13 = (1101)_2 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1.$$

The exponent rule $a^{u+v} = a^u a^v$ now gives

$$a^{13} = a^8 a^4 a.$$

More generally, compute $a, a^2, a^4, a^8, \dots, a^{2^k}$ by **repeated squaring**, then multiply the powers corresponding to the 1 bits of n .

3.5.1 Example 1: Compute 3^{13}

First build the powers of 3 by squaring the previous value:

Bit position i	Power 2^i	Value 3^{2^i}	Used in 13?
0	1	3	Yes
1	2	9	No
2	4	81	Yes
3	8	6561	Yes

Building the table takes three squarings: $3^2 = 9$, $9^2 = 81$, and $81^2 = 6561$. Even though the bit for 2 is zero, we still compute 3^2 to obtain the next power by squaring. Multiplying the three selected values takes two more multiplications:

$$3^{13} = 6561 \cdot 81 \cdot 3 = 531441 \cdot 3 = \boxed{1594323}.$$

The total is $3 + 2 = 5$ multiplications; the natural algorithm takes 12.

The recursive algorithm reaches the same answer in a different order. Its exponents descend as $13 \rightarrow 6 \rightarrow 3 \rightarrow 1$; on the way back up:

Exponent	Returned value	New multiplications
1	3	0
3	$(3 \cdot 3) \cdot 3 = 27$	2
6	$27 \cdot 27 = 729$	1
13	$(729 \cdot 729) \cdot 3 = 1594323$	2

Again the total is five. The intermediate numbers differ, but both methods use the binary structure of the exponent.

3.5.2 The Recursion Reads the Bits

Integer halving removes the last binary bit, while parity tells us which bit was removed:

$$(1101)_2 \longrightarrow (110)_2 \longrightarrow (11)_2 \longrightarrow (1)_2.$$

On the way back up, suppose the bits processed so far represent q , and the current result is a^q . Appending a bit b changes the exponent to $2q + b$. Thus we square the result, and if $b = 1$, multiply once more by a :

$$a^{2q+b} = (a^q)^2 a^b.$$

This is exactly the even/odd rule used by the recursive algorithm. It also gives an iterative version that reads the bits from left to right:

```

BinaryPower(a, n):                                // n >= 1
    write n in binary
    result = a                                     // the leading bit is 1
    for each remaining bit b, from left to right:
        result = result * result
        if b == 1:
            result = result * a
    return result

```

For $n = 0$, return 1 separately. After each processed bit, **result** is a raised to the exponent represented by the processed prefix. The equation above explains why each update preserves this property.

3.5.3 Example 2: Compute 2^{10}

Since $10 = (1010)_2$, start with the leading bit 1 and **result**= 2.

Next bit	Prefix	Update	New multiplications
0	$(10)_2 = 2$	$2^2 = 4$	1
1	$(101)_2 = 5$	$4^2 \cdot 2 = 32$	2
0	$(1010)_2 = 10$	$32^2 = 1024$	1

Thus $2^{10} = 1024$, using four multiplications instead of nine. From the repeated-squaring viewpoint, compute $2^2 = 4$, $2^4 = 16$, and $2^8 = 256$, then multiply $256 \cdot 4 = 1024$. This also uses three squarings and one extra multiplication.

3.5.4 An Exact Count from the Binary Digits

Let $s(n)$ be the number of 1 bits in the binary expansion of $n \geq 1$. After the leading 1, there are $\lfloor \log_2 n \rfloor$ bits left to process. Each requires one squaring, and each of the remaining $s(n) - 1$ ones requires one additional multiplication by a . Therefore

$$\boxed{\text{actual multiplications} = \lfloor \log_2 n \rfloor + s(n) - 1 \quad (n \geq 1).}$$

The repeated-squaring table uses the same count: it takes $\lfloor \log_2 n \rfloor$ squarings to build the powers and $s(n) - 1$ multiplications to combine the selected entries, starting with the first selected entry rather than with 1.

For $13 = (1101)_2$, the formula gives $3 + 3 - 1 = 5$. For $10 = (1010)_2$, it gives $3 + 2 - 1 = 4$. Since $1 \leq s(n) \leq \lfloor \log_2 n \rfloor + 1$, the exact count is at most $2\lfloor \log_2 n \rfloor$, as in our upper-bound analysis. For instance, when $n = 1024 = 2^{10}$, only one bit is 1, so fast exponentiation uses just 10 multiplications, compared with 1023 for the natural algorithm.